

Bakalářské zkoušky (příklady otázek)

2026-06-22

1 Rozděl a panuj (společné okruhy)

Uvažme následující úlohu: Je dána matice A složená z $n \times n$ celých čísel. Každý její řádek i sloupec tvoří neklesající posloupnost. Chceme zjistit, zda se v této matici nachází 0. Matice je už načtena v paměti, čtení vstupu není potřeba řešit. Načtená vstupní data se nepočítají do prostorové složitosti algoritmu, který úlohu řeší.

Použijeme metodu Rozděl a panuj. Podíváme se na prostřední prvek matice. Pokud je nulový, uspěli jsme. Pokud je kladný, všechny prvky v pravé dolní čtvrtině matice jsou také kladné, takže stačí úlohu řešit rekurzivně ve zbývajících třech čtvrtinách. A pokud je prostřední prvek záporný, můžeme naopak vyloučit levou horní čtvrtinu.

Vaše úkoly:

1. Zapište algoritmus založený na uvedené myšlence. K zápisu můžete použít například jazyk Python, nebo pseudokód podobné úrovně detailu.
2. Analyzujte jeho časovou a prostorovou složitost v závislosti na n . Pokud k tomu používáte nějakou větu, uveďte její znění.
3. Navrhněte efektivnější algoritmus a analyzujte jeho časovou a prostorovou složitost.

Nástin řešení

1. Algoritmus může mít podobu rekurzivní funkce, která dostane levý horní roh a rozměry podmatice, v níž má hledat. V Pythonu ji můžeme zapsat takto:

```
def find(i, j, n, m):
    if n < 1 or m < 1:
        return None
    nn = n // 2
    mm = m // 2
    ci = i + nn
    cj = j + mm
    if A[ci][cj] == 0:
        return True
    elif A[ci][cj] < 0:
        return (find(i+nn, j, n-nn, mm) or
                find(i, j+mm, n, m-mm) or
                find(i+nn, j+mm, n-nn, m-mm))
    else:
        return (find(i, j, nn, mm) or
                find(i+nn, j, n-nn, mm) or
                find(i, j+mm, nn, m-mm))
```

2. Časová složitost je popsána rekurencí $T(n) = 3T(n/2) + \Theta(1)$ s okrajovou podmínkou $T(1) = \Theta(1)$. Jejím řešením podle Kuchařkové věty (viz Průvodce labyrintem algoritmů, kapitola 10.4) je $T(n) \in \Theta(n^{\log_2 3}) \subseteq \mathcal{O}(n^{1.59})$. Prostorová složitost je popsána rekurencí $S(n) = S(n/2) + \Theta(1)$, $S(1) = \Theta(1)$ s řešením $S(n) \in \Theta(\log n)$.
3. Jedna možnost je v každém řádku zvlášť binárně vyhledávat, čímž dosáhneme složitosti $\Theta(n \log n)$.

Nebo se můžeme podívat na levý dolní prvek matice. Pokud je kladný, jsou kladné všechny prvky v posledním řádku, takže můžeme celý poslední řádek smazat. Pokud je naopak záporný, můžeme smazat celý první sloupec. Těchto kroků provedeme nejvýše $2n$, než buď najdeme nulu, nebo matici zredukujeme na prázdnou. Jeden krok trvá konstantní čas a zabere konstantní prostor, takže celý algoritmus běží v čase $\Theta(n)$ a prostoru $\Theta(1)$.

Dodejme ještě, že asymptoticky rychleji to nejde – uvážíme-li matici, která má na vedlejší diagonále a všude nad ní samé -1 a pod diagonálou 1 . Kteroukoliv -1 na diagonále můžeme změnit na 0 , aniž bychom porušili uspořádání. Takže každý korektní algoritmus musí přechít všech n prvků diagonály.

2 Průchod spojovým seznamem (společné okruhy)

Předpokládejme 16bitový big-endian RISC procesor, který podporuje instrukce uvedené v tabulce níže. Registry procesoru jsou 16bitové, virtuální adresový prostor dovoluje adresovat právě pomocí 16bitových adres.

Procesor má k dispozici 10 obecných registrů R0 až R9, které uchovávají unsigned integer o velikosti 16 bitů (další registry nejsou pro naši úlohu potřeba; pro řešení využijte pouze registry, pomocné výsledky neukládejte do paměti). V celé otázce pracujeme jen s virtuálními adresami, překlad na fyzické adresy není potřeba vůbec řešit.

Zároveň máme k dispozici strukturu pro počítání frekvencí čísel využívající triviální spojový seznam. V celé otázce předpokládáme korektně vytvořený spojový seznam, tj. není potřeba detektovat cyklus, špatné zarovnání v paměti apod.

```
typedef struct freq freq_t;
struct freq {
    uint16_t number;
    uint16_t count;
    freq_t *next;
};
// next je u posledního prvku nastaveno na 0
...

freq_t *head;
```

RD označuje cílový registr, RS pak značí zdrojové registry. memory je pro přístup do paměti na dané adrese.

LI RD, imm	RD := imm (načtení konstanty)
LW RD, offset(RS)	RD := memory[RS + offset]
ADD RD, RS1, RS2	RD := RS1 + RS2
OR RD, RS1, RS2	RD := RS1 bitwise or RS2
SW RS1, offset(RS2)	memory[RS2 + offset] := RS1
BEZ RS, label	if RS == 0: jump to label
BNEZ RS, label	if RS != 0: jump to label
JMP label	jump to label

imm a offset jsou konstanty (nelze použít registr!)

Proměnná `head` je uložena v paměti na adrese `0x900a` (tj. v tomto místě je pointer na první skutečný prvek seznamu).

Část A: vyznačte orámováním a šipkami jednotlivé prvky seznamu v následujícím hexdumpu paměti. (Hexdump je ve stejné podobě k dispozici i na další stránce, zřetelně vyznačte, který obsahuje finální řešení.)

```
8ff0: 45 2f 90 11 8f 0f 90 42 01 23 45 90 42 42 90 90
9000: 89 e2 26 90 05 ff 00 de ad 00 90 10 90 14 00 00
9010: 00 01 00 05 90 42 ca fe ca fe 42 90 30 be ef 35
9020: 45 01 90 54 42 53 01 87 aa bb 01 23 45 67 90 24
9030: 12 02 08 90 17 44 ab cd 90 48 20 17 00 5f 00 00
9040: 77 88 01 23 00 f0 90 3a 5e d3 41 90 48 90 4c 00
9050: 39 4a 85 90 24 48 89 00 90 10 90 00 42 23 ca fe
```

Část B: doplňte posloupnost instrukcí (v pravém sloupečku) tak, aby v registru R3 bylo uloženo celkové množství prvků, tj. součet položek `count` ve výše uvedené struktuře. Okomentujte využití jednotlivých registrů (tj. jak by pravděpodobně byly pojmenované odpovídající proměnné).

Váš kód musí být napsán obecně pro libovolný platný seznam. Tj. doplněný kód by neměl obsahovat žádné pevné adresy, kromě adresy hlavy seznamu (která je již předpřipravená). Váš kód nemá řešit případné přetečení při sčítání prvků nebo kontrolu přístupu do paměti.

V případě skoků využijte pojmenovaná návěští (např. `loop_start`).

```
LI R1, 0x900a
LW R2, 0(R1)
LI R3, 0
// <-- sem doplňte vhodné instrukce
end:
// v registru R3 je součet položek count
```

(Druhá strana obsahuje tentýž instrukční blok, finální instrukce doplňte do něj.)

Student 666. Řešení můžete vyznačit na této stránce, která reprodukuje hexdump a kód ve větším formátu.

```
8ff0: 45 2f 90 11 8f 0f 90 42 01 23 45 90 42 42 90 90
9000: 89 e2 26 90 05 ff 00 de ad 00 90 10 90 14 00 00
9010: 00 01 00 05 90 42 ca fe ca fe 42 90 30 be ef 35
9020: 45 01 90 54 42 53 01 87 aa bb 01 23 45 67 90 24
9030: 12 02 08 90 17 44 ab cd 90 48 20 17 00 5f 00 00
9040: 77 88 01 23 00 f0 90 3a 5e d3 41 90 48 90 4c 00
9050: 39 4a 85 90 24 48 89 00 90 10 90 00 42 23 ca fe
```

```
8ff0: 45 2f 90 11 8f 0f 90 42 01 23 45 90 42 42 90 90
9000: 89 e2 26 90 05 ff 00 de ad 00 90 10 90 14 00 00
9010: 00 01 00 05 90 42 ca fe ca fe 42 90 30 be ef 35
9020: 45 01 90 54 42 53 01 87 aa bb 01 23 45 67 90 24
9030: 12 02 08 90 17 44 ab cd 90 48 20 17 00 5f 00 00
9040: 77 88 01 23 00 f0 90 3a 5e d3 41 90 48 90 4c 00
9050: 39 4a 85 90 24 48 89 00 90 10 90 00 42 23 ca fe
```

```
LI R1, 0x900a
LW R2, 0(R1)
LI R3, 0
```

```
end:
// v registru R3 je soucet polozek count
```

Nástin řešení V hexdumpu je na adrese 0x900a adresa prvního („skutečného“) prvku (tj. 0x9010), na adrese 0x9014 je pak adresa následujícího atd. (celkem 3 prvky, 5×1 , 240×291 a 95×8215).

U instrukcí je potřeba začít kontrolou na prázdný seznam. Registr R3 obsahuje od začátku průběžný součet, R2 je adresa aktuálního prvku (na konci seznamu bude obsahovat 0); do pomocného registru R4 ukládáme položku count právě zpracovávaného prvku.

```
LI R1, 0x900a
LW R2, 0(R1)
LI R3, 0
loop:
BEZ R2, end
LW R4, 2(R2)
ADD R3, R3, R4
LW R2, 4(R2)
JMP loop
end:
// v registru R3 je součet položek count
```

3 Lineární algebra, matice (společné okruhy)

Uvažujme matice

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 3 & 1 \\ 1 & 1 & 3 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 2 & 3 \\ 1 & 2 & 2 \\ 3 & 3 & 1 \end{pmatrix}.$$

1. Najděte bázi prostoru

$$U = \{x \in \mathbb{R}^3 : Ax = Bx\}.$$

2. Vypočítejte $\det(A^2 B A^T B^{-1})$.
3. Rozhodněte, zda je matice $A - B$ diagonalizovatelná. Rozklad hledat nemusíte, ale jeho existenci zdůvodněte.

Nástin řešení

1. Musíme najít jádro matice

$$A - B = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 1 & -1 \\ -2 & -2 & 2 \end{pmatrix}.$$

Bázi jádra tvoří např. vektory $(1, 0, 1)^T$, $(0, 1, 1)^T$.

2.
$$\det(A^2 B A^T B^{-1}) = \det(A)^2 \det(B) \det(A) \det(B)^{-1} = \det(A)^3 = (-5)^3 = -125.$$

3. Jádro $A - B$ má dimenzi 2 a stopa $A - B$ je 3. Tudíž 0 je vlastní číslo s algebraickou i geometrickou násobností 2 a zbývajícím vlastním číslem je 3 (s násobností 1). Z toho plyne existence báze $\mathbb{R}^3$ složené z vlastních vektorů $A - B$ a ta je tím pádem diagonalizovatelná.

4 Interval spolehlivosti pro střední hodnotu (společné okruhy)

1. Nechtě $X_1, \dots, X_n$ jsou nezávislá pozorování z normálního rozdělení $N(\mu, \sigma^2)$, kde σ známe, μ ne. Napište 95% interval spolehlivosti (konfidenční interval) pro μ .
2. Měříme dobu běhu programu. Předpokládejme, že doba jednoho běhu má normální rozdělení se známou směrodatnou odchylkou $\sigma = 12$ ms. Z $n = 36$ nezávislých měření vyšel průměr $\bar{X}_{36} = 105$ ms. Spočítejte 95% interval spolehlivosti pro skutečnou střední dobu běhu.

3. Vysvětlete slovně, co znamená označení 95% interval spolehlivosti.

Možná se vám budou hodit následující tabulky, kde Φ je distribuční funkce standardního normálního rozdělení $N(0, 1)$.

x	-2.0	-1.5	-1.0	-0.5	0.0	0.5	1.0	1.5	2.0
$\Phi(x)$	0.02	0.07	0.16	0.31	0.5	0.69	0.84	0.93	0.98

p	0.9	0.95	0.975	0.99	0.995
$\Phi^{-1}(p)$	1.2816	1.6449	1.96	2.3263	2.5758

Nástin řešení

1. Protože

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1),$$

dostaneme přibližně, respektive při splnění normálního modelu přesně, interval

$$\bar{X} \pm \Phi^{-1}(0.975) \frac{\sigma}{\sqrt{n}} = \left[\bar{X} - \Phi^{-1}(0.975) \frac{\sigma}{\sqrt{n}}, \bar{X} + \Phi^{-1}(0.975) \frac{\sigma}{\sqrt{n}} \right].$$

2. Dosadíme

$$105 \pm 1.96 \frac{12}{\sqrt{36}} = 105 \pm 1.96 \cdot 2 = 105 \pm 3.92.$$

Interval je tedy $[101.08, 108.92]$ ms, nebo po zaokrouhlení $[101, 109]$ ms.

3. Parametr μ chápeme jako pevnou neznámou hodnotu a interval jako náhodný, protože závisí na náhodném vzorku. Kdybychom stejný postup opakovali mnohokrát, přibližně 95% takto získaných intervalů by skutečnou hodnotu μ obsahovalo.